

From Mined Fixes to Gated Repair: Closed-Loop Defect Synthesis for CIRCT

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla

International Institute of Information Technology, Hyderabad

{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

Abstract—Software bugs in hardware compilers like CIRCT risk injecting silent defects into physical chips. AI repair agents can fix known issues, but they are constrained by the slow trickle of human-reported bugs. We present a closed loop, built on the CHIA agent framework, that mines historical CIRCT commits to generate its own defect supply: a model learns the mistake pattern behind each of 187 past fixes and synthesises novel, bug-exposing inputs. The loop offloads building, judging, test reduction and deduplication entirely to deterministic tools, hands patch generation to CHIA’s existing repair chain, and reserves human effort exclusively for final filing approval. It restores the compiler’s internal assertions, compiled out of CHIA’s published image, in 536 of 555 objects, so defects become observable where none could fire before. A registered two-hour campaign then found three new lowering bugs, each reproduced, minimised, validated and unmatched against 2,652 issues, held for a named human to file, while a mutation baseline over 1,154 inputs found none; every model call of the project cost USD 66 at list price.

I. INTRODUCTION

CIRCT is an open-source compiler for hardware. A designer writes how a circuit should behave, in FIRRTL (the form the Chisel hardware language produces) or in SystemVerilog, and CIRCT lowers it to Verilog, which synthesis and layout tools turn into a chip. It is built on MLIR, the LLVM project’s compiler framework, and inherits LLVM’s way of testing: a regression test carries the command to run and the output to expect, run by a tool called `lit`. A bug that crashes CIRCT is an inconvenience, seen and worked around; one that makes it emit wrong Verilog is not: the output is well formed, passes the tools below it, and can reach fabrication unnoticed.

CHIA [1] is the agent framework this hackathon is built on. Its CIRCT issue solver chains six phases, from assessing an issue to reproducing, fixing, verifying against the whole test suite, repairing regressions and writing the pull request; on 16 CIRCT issues it fixed five, three of them merged.

Its supply is the constraint. Over the past year CIRCT received 82 bug-labelled issues and closed 65; what does not drain is the backlog of 101 open ones, at a median age of 911 days. The scarce resource is maintainer attention, and an agent that empties the queue faster creates none. What would help is a bug that arrives already minimised, already checked against what is known, already carrying a candidate fix, from a source that files nothing else. Past fixes are the unspent information: a commit that corrects one wrong assumption inside a pass usually leaves the same assumption at other sites, and CIRCT ships no fuzzing infrastructure to find them.

This paper describes a loop that manufactures its own supply and then withholds most of it.

II. THE LOOP

Fig. 1 names the stages and marks the ones the agent runs; tools decide everything else. One model serves every agent stage, Gemini 3.8 Flash through CHIA’s Vertex backend, and no stage can build a client for it unless one environment variable is set for that run, so a test or a dry run cannot reach the model.

Read the seed (1). A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, with its diff and the test its author added or changed. The agent reads both, states in one sentence the root-cause class the fix corrected, and names other sites where the same assumption still stands.

Write probing inputs (2). The seed’s own test decides the language, because its command line names the tool through which the bug was reachable: `circt-opt`, `firtool` or the SystemVerilog front end. About 127 of the 187 seeds sit in code no FIRRTL input can reach, so a FIRRTL-only generator would miss most of the corpus. The agent reads the compiler’s source and writes each candidate through tools of its own, and runs no command.

Build (3). CIRCT compiles against one pinned commit of LLVM and MLIR, and Section III explains how the loop avoids building those from source. The build compiles in CIRCT’s assertions, the internal checks its own authors wrote, which stop the compiler the moment an assumption breaks.

Judge (4). An oracle is the rule that decides whether an input found a bug. The primary one admits no argument: the compiler crashed, or an assertion fired. A second runs the same design through CIRCT’s simulator and through Verilator, and treats disagreement as suspicious; it reports only, because the two disagree routinely over uninitialised and reset signals.

Reduce (5). A crashing input is usually large and mostly irrelevant to the crash. `circt-reduce`, which CIRCT ships, deletes parts of it and keeps each deletion while the failure survives; a textual reducer takes over where that tool dies.

Triage and report (6). Many failures are one failure, and deciding which are the same is mechanical: a fingerprint is the assertion text with its source location, or, for a crash, the symbolised frames left once the crash handler’s prologue is stripped. Survivors are matched against a local mirror of CIRCT’s open and closed issues and against commits made

Tested on real failures. The oracle and its fingerprints were calibrated on real output: six past CIRCT bugs, rebuilt at the commit before their fix and recorded with input, argument vector and error output, in three classes across two SDK releases, 14s to 601s of build each. Two defects in the fingerprinting rules appeared only there: the name normaliser erased every frame in an anonymous namespace, which is where CIRCT’s passes live, and the scope test read an inlined frame instead of the physical one, putting three of the four real bugs outside the repair stage’s reach. Both are fixed.

IV. CONTROLS

The experiment the loop embodies asks whether reading a past fix makes an agent better at finding new bugs than mutating the same tests at random. Two arms share the harness, oracles, reducer and gate, and only the generator differs: the seeded arm is stages 1 and 2, and the mutation arm rewrites the same seed tests syntactically without seeing the diff or the root-cause class, following Mut4All’s design for mutators synthesised by a language model [2]. Both arms spend the same budget, a fixed window of wall-clock seconds, one after the other on the same cluster at the same concurrency, and the artefact renders one headline: distinct bugs a CIRCT maintainer confirmed by labelling, commenting on or fixing them. Our oracle agreeing with itself produces a candidate, not a bug.

Pre-registration. Every threshold lives in one budget file, and a run is registered by an annotated git tag on the commit that lands that file; the loop refuses a file no tag names and a mutator set committed after the tag, so no threshold can be chosen after the fact. Safety caps bound damage rather than effort: 2,000 generated inputs per arm per day, 3 filings per day and 10 per run, 60 seconds and 4 GiB per probe, and a hard cap on model spending that stops both arms. Nothing is filed until a human has posted the method to the project’s forum, and issues marked as good first issues are refused.

Deduplication. Two candidates are one bug when a single primary fingerprint is equal. On a hand-labelled set of 22 candidate pairs the rule collides on 1 of the 11 distinct pairs and falsely merges 1 of the 11 duplicate pairs; both rates are printed beside the headline, since a collision inflates that count and a false merge deflates it.

Known issues. A mirror of the tracker answers whether a candidate is already reported. GitHub stops paginating at its hundredth page, so the mirror walks newest first, then oldest first, and unions the two: the whole tracker, 2,652 issues, 1,795 closed and 857 open, costs 198 requests and 200 seconds.

Contamination. CIRCT’s history sits in the training data of any model we can use, so the agent may reproduce a fix it memorised. Every candidate is screened against fixes made after its seed, at the level of the symbols the seed’s fix touched: that screen flags 77% of the corpus where the same screen at file level flags 96%. A real crash is a real crash, so the flag qualifies a candidate rather than excluding it.

V. RESULTS

Table II is the registered campaign: two hours per arm, the seeded arm as two shards in parallel on the same cluster, USD

TABLE II
THE CAMPAIGN, FROM THE LOOP’S DATABASE. THE SEEDED ARM IS BOTH TWO-HOUR SHARDS TOGETHER; THE MUTATION ARM EXHAUSTED ITS SEEDS IN 13 MINUTES, 87 OF THEM SKIPPED BECAUSE THEIR OWN TEST NO LONGER PARSES AT THE BUILD COMMIT

Measure	seeded	mutation
Seeds probed	9	96
Probes written	134	1,154
Rejected by the parser	72	400
Compiler’s output rejected by its verifier	10	0
Assertion failures	5	0
Candidates	15	0
Distinct new bugs through the gate	3	0
Duplicates of those	9	0
Fixed after the pin, held for review	2 (+2 dup.)	0
Filed	0, awaiting approval	0
Model turns; billed over authorised, mean	55; 0.18	0
Spend, USD at list price; cached share of input	44.5; 68%	0

100 cap per job. It was started three times. We stopped the first two at 32 and 45 minutes for defects the live run exposed, a feedback line cut before its error text and a scheduling deadlock between a shard’s generator and the gate; the third ran both shards to their windows. Every number is read from the loop’s database after the stored probes were re-judged under the two corrections described below; no model turn was re-run for that.

Three new bugs, one fix family. All three came from one seed, the August fix for out-of-bounds `moore.extract` lowering, and sit beside it: an extract at the most negative 32-bit offset makes the lowering build an array of zero elements and trips the assertion at `HWOps.cpp:1995`; lowering a dynamic extract emits `comb.extract` with an aggregate result type; lowering an extract of a reference emits `llhd.sig.extract` with an aggregate reference type. The last two are the compiler emitting IR its own verifier rejects. Each reproduced in a fresh process on another worker, was minimal at the reducer’s fixpoint, at most five lines, valid, and matched neither an issue nor a commit since the pin; the model wrote the three reports, and they are held for a named human.

The oracle had to learn a class. Ten of the seeded arm’s 82 rejected probes were the compiler’s fault, not the model’s: the input alone parses and verifies, and the diagnostic has the shape of MLIR’s generated verifier. Both conditions are needed, since a pass may complain legitimately about a valid input, such as two top-level modules. None of the mutation arm’s 400 rejections qualified: a mutated test that fails to parse is broken syntax, while the model’s inputs parse and exercise the lowering.

Dedup needed four corrections, each found on a real candidate. A commit after the pin counted as a fix if it touched the crash’s file, so an LLVM version bump touching 87 files silenced the first find; it now has to touch a frame symbol. A generic MLIR assertion text matched an open issue; it now also needs a CIRCT frame token in the issue. A verifier failure matched an issue on the op name alone; it now needs the constraint too. And two candidates sharing a fingerprint were each the other’s duplicate, so thirteen candidates had no representative; the duplicate is now of the earliest. Because every verdict is recomputed from stored evidence, correcting a rule re-judges every stored candidate without a model turn.

Money. Fifty-five turns billed 0.18 of what the guard authorised before the first call, never more than 0.41; 68% of the 55.6 million input tokens were served from Google’s cache, so the bill is below the list price the ledger records. Eight capped pilots preceded it for USD 13 in total, each retiring one defect in the model-turn path; the first found a real assertion on its first seed and withheld it as a duplicate of open issue 8508. The two mutator syntheses cost USD 0.49 each, and naming the regular-expression engine in the prompt raised the kept mutators from 11 of 22 to 25 of 25.

Repair. Three attempts on the assertion. The first two never reached a patch: the chain’s reproduce step could not confirm the loop’s own script, which, we found, handed the tool two input files, our defect, not the chain’s. With that fixed the chain reproduced the failure, wrote a one-line change, rebuilt CIRCT and ran its test gate, and was writing up when the 45-minute wall we set ended it; the change was a debug print, not a fix. A fix is not claimed.

VI. RELATED WORK

SWE-bench [3] scores agents on real GitHub issues and CHIA’s issue solver [1] applies the pattern to a hardware compiler; both draw from a queue, and we fill one. Fuzzers fill queues without repairing or withholding: CSmith [4] for C; for MLIR, MLIRod [5], Nüwa [6], DESIL [7] and FLEX, 80 unknown bugs in 30 days [8]; our mutation arm follows Mut4All’s synthesised mutators [2]. AFuzz [9] is the origin of our method, an agent testing where else a reference bug’s cause could live, 40 bugs in V8 in a month; we apply it behind a gate. Delta debugging [10] is stage 5’s algorithm. Abstain-and-Validate [11], llvm-harness [12] and CodeRover-S [13] make a repair agent more selective; our gate sits downstream and discards by default.

VII. ARTEFACT AND LIMITATIONS

We release the loop as CHIA nodes and tools, so swapping the generator for the mutation arm is one node substituted. Its two halves meet at a frozen contract of seven record types and one function call, version 2.4, tested separately against recorded examples; the suite is 736 tests, 686 of which need no cluster, network or model. On CHIA’s cluster layer the loop’s five containers come up in 38 seconds and, with a recorded generator, drive 16 probes through both arms in 7.4 seconds. Four things go upstream to CHIA: the Dockerfile for the assertions-on image, its build workflow, three helpers for its CIRCT module, and the nineteen-line branch that lets its repair chain select our backend. All of it, with the seed corpus and every input, verdict, reduced case, transcript and gate decision, is at https://github.com/DevAbhinav-23/CHIA_Hackathon_2k26.

The second oracle’s harness generators emit, from one design, a CIRCT simulation harness and a Verilator testbench printing identical 112-line sequences, and one changed constant makes them disagree at a named cycle and signal.

Limitations. Nothing has been filed at the time of writing; filing is the approver’s act and follows the forum post. Model

time across every live run totals USD 66 at list price, and the repair chain’s own model use is not observable, so its one attempt is counted at its authorisation. The second oracle applies to 6 of the 187 seeds. The 187 rest on a subject-keyword filter, no diffs read to confirm each is a bug fix; of seven mining attempts, one seed did not fail at its parent. The contamination screen misses a sibling fixed in a commit whose subject does not name it. Calibration by re-injecting known bugs is not possible on one prebuilt image: no seed’s parent shares its LLVM pin. The image is built but not published, so a report names its content hash rather than a registry digest.

Scope. The loop does not repair wrong-output bugs, because the oracle that would find them is report-only, and it does not replace maintainers: it converts the human event from writing a bug report into reading one.

ACKNOWLEDGEMENTS

We thank the CHIA developers and the A³ hackathon organisers. *AI-assistance disclosure*. Claude (Anthropic) and Gemini (Google) were used for drafting, code generation and revision; the human authors made every claim, decision and number their own and are responsible for the whole paper.

REFERENCES

- [1] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.
- [2] B. Wang *et al.*, “Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports,” *arXiv preprint arXiv:2507.19275*, 2025.
- [3] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [4] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.
- [5] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, “Fuzzing MLIR compiler infrastructure via operation dependency analysis,” in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024, pp. 1287–1299.
- [6] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, “Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation,” in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.
- [7] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, “DESIL: Detecting silent bugs in MLIR compiler infrastructure,” *arXiv preprint arXiv:2504.01379*, 2025.
- [8] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, “Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR,” in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.
- [9] J. Park and I. Yun, “Agentic fuzzing: Opportunities and challenges,” *arXiv preprint arXiv:2605.10074*, 2026.
- [10] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [11] J. Cambroner *et al.*, “Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026.
- [12] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, “Agentic harness for real-world compilers,” *arXiv preprint arXiv:2603.20075*, 2026.
- [13] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, “Fixing security vulnerabilities with agentic AI in OSS-Fuzz,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 374–384.